

DANH SÁCH ĐỀ TÀI GIỮA KỲ

BUỔI 6: Chương 3 – Thiết kế dữ liệu (2)

Nhóm 1: NoSQL và mô hình dữ liệu phi quan hệ

Lý thuyết & khái niệm cần tìm hiểu

- Khái niệm NoSQL và 4 nhóm chính: Document, Key-Value, Column-family, Graph
- CAP Theorem (Consistency – Availability – Partition tolerance) và đánh đổi giữa các tính chất
- So sánh mô hình dữ liệu: schema cứng (SQL) và schema linh hoạt (NoSQL)
- Ví dụ công cụ thực tế: MongoDB, Redis, Cassandra, Neo4j

Ứng dụng cụ thể vào đề án cuối kỳ

Xác định trong đề án cuối kỳ có phần dữ liệu nào ít ràng buộc quan hệ chặt (log hoạt động, giỏ hàng tạm, cache phiên làm việc...), đề xuất kiến trúc polyglot persistence (dùng SQL cho dữ liệu lõi + NoSQL cho phần phù hợp), hoặc lý giải rõ vì sao SQL thuần vẫn là lựa chọn đúng cho đề tài của nhóm.

Tài liệu tham khảo

- Ian Sommerville, Software Engineering, 9th ed. – chương Database design
- Silberschatz, Korth, Sudarshan, Database System Concepts, 7th ed. – chương NoSQL Systems
- MongoDB Official Documentation – “NoSQL Explained” (mongodb.com/nosql-explained)
- Martin Fowler – “NoSQL Distilled” (martinfowler.com)

Nhóm 2: ORM (Object-Relational Mapping) – Ánh xạ đối tượng sang CSDL quan hệ

Lý thuyết & khái niệm cần tìm hiểu

- Object-Relational Impedance Mismatch – vì sao thế giới hướng đối tượng và CSDL quan hệ không khớp tự nhiên
- Các mẫu ánh xạ: Active Record, Data Mapper
- Lazy loading vs Eager loading, khái niệm migration
- Công cụ thực tế: Entity Framework, Hibernate, Sequelize, Prisma (chọn theo công nghệ đề án)

Ứng dụng cụ thể vào đề án cuối kỳ

Chọn một ORM phù hợp với công nghệ đề án cuối kỳ, minh họa ánh xạ 2-3 bảng chính từ ERD/Class Diagram của đề án sang code thực tế, chỉ rõ lợi ích (tốc độ phát triển) và hạn chế (hiệu năng) so với viết SQL thuần.

Tài liệu tham khảo

- Martin Fowler, Patterns of Enterprise Application Architecture – chương Object-Relational Mapping patterns

- Roger S. Pressman, Kỹ nghệ Phần mềm
 - Microsoft Docs – Entity Framework Core Documentation, hoặc Hibernate ORM User Guide
-

BUỔI 7: Chương 4 – Thiết kế giao diện (1)

Nhóm 3: Design System & UI Kit hiện đại (Material Design, Apple HIG)

Lý thuyết & khái niệm cần tìm hiểu

- Design System là gì: design tokens, component library, style guide
- So sánh Material Design (Google) và Human Interface Guidelines (Apple)
- Lợi ích: tính nhất quán, tốc độ phát triển, dễ mở rộng khi làm việc nhóm đông

Ứng dụng cụ thể vào đồ án cuối kỳ

Chọn một design system, áp dụng bảng màu/typography/component chuẩn cho ít nhất 2 màn hình chính của đồ án cuối kỳ; trình bày đối chiếu giao diện trước và sau khi áp dụng.

Tài liệu tham khảo

- material.io/design – Material Design 3 Guidelines
 - developer.apple.com/design/human-interface-guidelines
 - Dương Anh Đức, Giáo trình phân tích & thiết kế hướng đối tượng bằng
-

Nhóm 4: Công cụ Wireframe/Prototype thực tế: Figma và quy trình thiết kế cộng tác

Lý thuyết & khái niệm cần tìm hiểu

- Phân biệt wireframe (low-fidelity) – mockup (visual) – prototype (interactive)
- Quy trình thiết kế cộng tác nhiều người: component, auto layout, design handoff cho lập trình viên

Ứng dụng cụ thể vào đồ án cuối kỳ

Dùng prototype có thể click-through cho ít nhất 3 luồng chức năng chính (use case) của đồ án cuối kỳ trước khi viết code, dùng để rà lại yêu cầu trước khi cài đặt.

Tài liệu tham khảo

- Figma Help Center – “Guide to prototyping”
 - Ian Sommerville, Software Engineering – chương Requirements engineering
 - Don Norman, The Design of Everyday Things
-

BUỔI 8: Chương 4 – Thiết kế giao diện (2)

Nhóm 5: Responsive Design & Accessibility

Lý thuyết & khái niệm cần tìm hiểu

- Responsive design: breakpoint, fluid grid, mobile-first
- Accessibility: chuẩn WCAG 2.1, contrast ratio, hỗ trợ screen reader, điều hướng bằng bàn phím

Ứng dụng cụ thể vào đồ án cuối kỳ

Kiểm tra và điều chỉnh giao diện đồ án cuối kỳ chạy tốt trên ít nhất 2 kích thước màn hình; áp dụng tối thiểu 3 tiêu chí WCAG cơ bản (độ tương phản màu, alt text cho ảnh, focus indicator khi dùng bàn phím).

Tài liệu tham khảo

- W3C Web Accessibility Initiative – WCAG 2.1 Quick Reference ([w3.org/WAI/WCAG21/quickref](https://www.w3.org/WAI/WCAG21/quickref))
- web.dev (Google) – “Responsive Web Design Basics”
- Nielsen Norman Group – “Accessibility for UX” (nngroup.com)

Nhóm 6: Usability Testing, thiết kế kịch bản kiểm thử người dùng

Lý thuyết & khái niệm cần tìm hiểu

- Các bước usability testing: xác định mục tiêu, viết kịch bản, chọn người test, quan sát, phân tích kết quả
- Kiểm thử định tính vs định lượng; quy tắc 5 người dùng phát hiện ~85% lỗi (Nielsen)

Ứng dụng cụ thể vào đồ án cuối kỳ

Thiết kế 1 kịch bản test thật với 3-5 người dùng (bạn học khác nhóm) cho prototype/sản phẩm đồ án; ghi nhận, phân loại lỗi phát hiện được và đưa vào báo cáo cuối kỳ như minh chứng cải tiến giao diện.

Tài liệu tham khảo

- Jakob Nielsen – “Usability Testing 101” (nngroup.com)
- Steve Krug, Don't Make Me Think
- Roger S. Pressman, Kỹ nghệ Phần mềm

BUỔI 9: Chương 5 – Thiết kế xử lý (1)

Nhóm 7: Design Patterns cơ bản và mối liên hệ với nguyên lý SOLID

Lý thuyết & khái niệm cần tìm hiểu

- Định nghĩa design pattern, 3 nhóm: Creational / Structural / Behavioral
- Đi sâu 3-4 pattern phổ biến với đồ án sinh viên: Singleton, Factory, Observer, MVC
- Mỗi pattern hiện thực nguyên lý SOLID nào (VD: Factory ~ Open/Closed, Observer ~ Dependency Inversion)

Ứng dụng cụ thể vào đồ án cuối kỳ

Xác định 2-3 vị trí trong class diagram/kiến trúc của đồ án cuối kỳ có thể áp dụng pattern cụ thể; giải thích rõ lý do chọn pattern đó thay vì viết code trực tiếp.

Tài liệu tham khảo

- Gamma, Helm, Johnson, Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software (GoF)
- refactoring.guru/design-patterns
- Robert C. Martin, Clean Architecture
- Dương Anh Đức, Giáo trình phân tích & thiết kế hướng đối tượng bằng UML (tài liệu [2] trong đề cương)

Nhóm 8: Từ Class Diagram đến Code – sinh mã tự động & Reverse Engineering

Lý thuyết & khái niệm cần tìm hiểu

- Forward engineering (từ UML sinh code) và Reverse engineering (từ code dựng lại UML)
- Công cụ hỗ trợ: Paradigm Community Edition, PlantUML, dbdiagram.io
- Lợi ích và giới hạn của việc sinh mã tự động

Ứng dụng cụ thể vào đồ án cuối kỳ

Dùng Paradigm Community Edition hoặc PlantUML để sinh khung code từ class diagram của đồ án cuối kỳ; so sánh với code hoàn chỉnh nhóm tự viết để chỉ rõ phần nào công cụ sinh được, phần nào vẫn cần lập trình viên bổ sung logic.

Tài liệu tham khảo

- Ian Sommerville, Software Engineering – chương Software design and modelling
- Roger S. Pressman, Kỹ nghệ Phần mềm (tài liệu [3] trong đề cương)

BUỔI 10: Chương 5 – Thiết kế xử lý (2)

Nhóm 9: Kiến trúc phần mềm phổ biến: MVC vs MVVM vs Layered Architecture

Lý thuyết & khái niệm cần tìm hiểu

- Định nghĩa và so sánh MVC, MVVM, Layered/Clean Architecture
- Luồng dữ liệu trong mỗi kiến trúc; ưu/nhược điểm
- Kiến trúc nào phù hợp loại ứng dụng nào (web MVC, desktop/mobile MVVM, hệ thống lớn Layered)

Ứng dụng cụ thể vào đồ án cuối kỳ

Vẽ sơ đồ kiến trúc thực tế nhóm dùng cho đồ án cuối kỳ, chỉ rõ tầng nào xử lý gì (UI – Business Logic – Data Access); đối chiếu với class diagram/sequence diagram đã thiết kế trước đó.

Tài liệu tham khảo

- Martin Fowler, Patterns of Enterprise Application Architecture
- Tài liệu chính thức của framework nhóm sử dụng (VD: Microsoft Docs – MVVM Toolkit, Laravel MVC Docs)
- Ian Sommerville, Software Engineering – chương Architectural design

Nhóm 10: Quản lý trạng thái (State Management) & xử lý bất đồng bộ

Lý thuyết & khái niệm cần tìm hiểu

- Khái niệm state trong ứng dụng và vấn đề khi nhiều thành phần giao diện cùng chia sẻ state
- Giải pháp phổ biến: Redux/Context API (web), ViewModel/LiveData (mobile)
- Lập trình bất đồng bộ: async/await, callback, Promise

Ứng dụng cụ thể vào đồ án cuối kỳ

Xác định 1-2 luồng xử lý trong đồ án cuối kỳ có thao tác bất đồng bộ (gọi API, truy vấn CSDL, upload file); thiết kế rõ trạng thái loading/error/success hiển thị cho người dùng.

Gợi ý câu hỏi phản biện

- Điều gì xảy ra nếu người dùng bấm nút 'Đặt hàng' hai lần liên tiếp trong lúc hệ thống đang xử lý? Nhóm xử lý race condition này thế nào?

Tài liệu tham khảo

- MDN Web Docs – “Asynchronous JavaScript”
- Tài liệu chính thức React/Redux hoặc framework nhóm dùng
- Ian Sommerville, Software Engineering – chương Software design

BUỔI 11: Chương 6 – Cài đặt và kiểm thử (1)

Nhóm 11: DevOps & CI/CD – Tự động hoá build/test/deploy

Lý thuyết & khái niệm cần tìm hiểu

- Khái niệm DevOps; CI (Continuous Integration) và CD (Continuous Deployment)
- Pipeline cơ bản: build → test → deploy; công cụ phổ biến (GitHub Actions, GitLab CI, Jenkins)
- Git workflow nhóm: branching, pull request, code review

Ứng dụng cụ thể vào đồ án cuối kỳ

Thiết lập 1 pipeline CI đơn giản cho repo đồ án cuối kỳ (VD: GitHub Actions tự động build/chạy test mỗi khi push code); áp dụng quy trình review code trước khi merge trong nhóm.

Tài liệu tham khảo

- GitHub Docs – “About continuous integration” / GitHub Actions Documentation

- Jez Humble & David Farley, Continuous Delivery
- Ian Sommerville, Software Engineering – chương Software development tools

Nhóm 12: AI hỗ trợ lập trình – Copilot, AI code review và chất lượng mã nguồn

mã nguồn' vừa học sang các công cụ AI hỗ trợ lập trình đang được dùng phổ biến trong ngành hiện nay.

Lý thuyết & khái niệm cần tìm hiểu

- Các công cụ AI hỗ trợ lập trình hiện nay: GitHub Copilot, Claude Code, Cursor... và cách chúng gợi ý code theo ngữ cảnh
- Lợi ích (tăng tốc độ phát triển) và rủi ro (code sai, hiểu nhầm, lệ thuộc, vấn đề bản quyền)
- Khái niệm AI code review hỗ trợ phát hiện bug/vi phạm coding convention

Ứng dụng cụ thể vào đồ án cuối kỳ

Nhóm thử dùng một công cụ AI để hỗ trợ viết một phần nhỏ của đồ án cuối kỳ, ghi nhận lại: phần nào AI làm tốt, phần nào AI sai/cần sửa, và nhóm đã kiểm chứng lại đoạn code đó thế nào trước khi dùng, đưa thẳng nội dung này vào báo cáo cuối kỳ như minh chứng dùng AI có trách nhiệm.

Tài liệu tham khảo

- GitHub Docs – “About GitHub Copilot”
- Anthropic – “Claude Code overview” (docs.claude.com)
- Robert C. Martin, Clean Code
- Roger S. Pressman, Kỹ nghệ Phần mềm

LƯU Ý CHUNG KHI CHẤM ĐIỂM

- Nộp outline/slide trước 2-3 ngày để giảng viên duyệt trước, tránh sai sót kỹ thuật lan ra cả lớp.
- Trong phần phản biện, giảng viên gọi ngẫu nhiên bất kỳ thành viên nào của nhóm trình bày trả lời
- Nếu nhóm có dùng AI hỗ trợ tìm hiểu/soạn nội dung, yêu cầu khai rõ trong slide đã dùng ở đâu và đã kiểm chứng lại như thế nào.